Proceeding of the 2015 IEEE
International Conference on Information and Automation
Lijiang, China, August 2015

Parallel Implementation of Low Light Level Image Enhancement Using CUDA

Peiyi Shen, Liang Zhang, Juan Song, Xilu Peng, Guangming Zhu and Yi Zhang
School of Software Engineering
Xidian University
Xi'an, Shaanxi Province, China
{*pyshen, liangzhang*}*@xidian.edu.cn*

Lukui Zhi
Shaanxi Province Public Security Department
Xi'an, Shaanxi Province, China
sxsgatkjc@163.com

Kang Yi
Xi'an Communication Institute
Xi'an, Shaanxi Province, China
penfangs@163.com

***Abstract*— Enhancement algorithms can make low light level images have a clear visual effect like the one captured during the daytime, but due to high complexity and generous computational cost, low light level image enhancement algorithms are usually difficult to meet real-time requirements which make it difficult to be widely used in practical application. For this situation, a parallel optimization algorithm of low light level image enhancement using CUDA is proposed. Enhancement algorithm based on de-hazing technique is used and on CPU-GPU heterogeneous platform the part of atmospheric light estimation which is not suitable for parallel computing is improved to obtain high parallelism degree. By comparing the performance of the algorithm on GPU with CPU, we indicate that the algorithm proposed has a significant improvement in execution speed while maintaining the visual effect of the traditional algorithm.**

***Index Terms*— low light level image enhancement, CUDA, parallel optimization, real-time**

I. Introduction

On low light conditions, images captured by image acquisition devices are nearly invisible which make it difficult to obtain the feature information. Low light level enhancement techniques can meet the need of applications such as video surveillance system, which can convert the low light images to the bright ones with more details. Enhancement algorithms for low light level images mainly contain the fusion based methods [1], the Retinex based methods [2, 3 and 4] and the de-hazing based methods [5, 6]. Dong etc. in [5] propose a low light video enhancement method using de-hazing technique based on dark channel prior by comparing the similarities between the inverted images with low light intensity and images with dense fog.

Although some low light level image enhancement algorithms achieve obvious processing effect, due to high computation complexity, algorithms are difficult to meet real-time requirements in practical application. In the researches of low light level image enhancement, few algorithms have been done for parallel computing. GPU has a higher performance than CPU in floating-point and parallel computing field. Running the parallel part of algorithms on GPU instead of CPU to make full use of system resources has been widely used in computationally intensive applications [7, 8 and 9]. CUDA (Compute Unified Device Architecture) is a parallel computing architecture of NVIDIA graphics processing units (GPUs) which can coordinate the serial logic tasks processing on CPU and the parallel tasks processing on GPU. GPU can process the complex calculations effectively and improve processing speed significantly.

In this paper, we propose a parallel approach of low light level image enhancement using CUDA. The de-hazing based method is used based on CPU-GPU heterogeneous architecture, and the whole enhancement algorithm is processed on GPU. We improve the part which is not suitable for parallel computing through the analysis of key steps in the traditional enhancement algorithm. By improving the parallelism degree, the proposed approach can achieve real-time processing effects for high-resolution images while maintaining the visual quality of enhancement results.

II. CUDA-Based Fundamental

CUDA is a unified device architecture of high-performance computing on GPU for general parallel computing purpose. The CUDA-based programming model focuses on the collaboration of CPU and GPU, which contains two parts: CPU side and GPU side [10]. CPU side as the host-side processes the serial part of programs which contains the program entry. GPU side as the device-side processes kernels which can be implemented in parallel. On CUDA programming model, kernels are organized into grids and grids are composed by a group of thread blocks executing together, which is called two-level parallelism. Each thread has private local memory and each thread block has shared memory where all the threads in one block can access it. Threads in one block has the same life cycle as the block which contains them and the number of threads in one block is limited. Threads in the same kernel can share the memory resources of their kernel. Each kernel executes with block as its unit and each block executes in parallel. Because of containing many large-scale matrix operations, image processing is well suitable for parallel processing. The number of threads can be defined according to the total number of image pixels, which can achieve the maximum parallelism degree.

978-1-4673-9104-7/15/$31.00 ©2015 IEEE

III. Enhancement Of Low Light Level Image Based On De-Hazing Technique

Dong etc. in [5] indicate that inverted low light level images have high similarity with the images with dense fog. Therefore we can treat the inverted images as foggy images and use de-hazing algorithm based on dark channel prior to process them and obtain the enhancement results by inverting the image again. The de-hazing-based low light level image enhancement algorithm mainly contains the following key steps:

A. Inverting input

$$I_{inv}^{c}(x) = 255 - I_{in}^{c}(x) \tag{1}$$

Where I_{in}^{c} is the input low light level image, I_{inv}^{c} is the inverted result, c indicates one of the three color channels (RGB) of images.

B. Computing dark channel

The dark channel of inverted image can be described as follow:

$$D^{dark}(x) = \min_{c\in\{r,g,b\}}\Big(\min_{y\in\Omega(x)}(I_{inv}^{c}(y))\Big) \tag{2}$$

Where D^{dark} is the dark channel image, I_{inv}^{c} is the inverted image, c indicates one of the three color channels (RGB) and Ω indicates a local patch centered at x.

C. Estimating atmospheric light

To estimate the atmospheric light, He etc. in [11] choose the 0.1% pixels which have the maximum intensity from the dark channel map, and then select the corresponding pixel value with the highest intensity in the original image from the 0.1% pixels selected from the dark channel map as the global atmospheric light value.

D. Estimating transmission

Zhang etc. in [6] proposed a method to estimate the transmission map using luminance map instead of the soft matting method proposed in [11]. The luminance-based method observably reduces the computational complexity in the transmission estimation step while maintaining the visual quality of enhancement results. Therefore we choose the luminance-based method to parallel processing on GPU. The transmission value is given by

$$t(x) = C - Y(x)/255 \tag{3}$$

where C is a constant used to minus the luminance map with a range of [1.06, 1.08], $Y(x)$ is the luminance map defined as follows:

$$Y(x) = 0.299 \times I_r(x) + 0.587 \times I_g(x) + 0.114 \times I_b(x) \tag{4}$$

E. De-hazing

$$J(x) = \frac{I_{inv}^{c}(x) - A}{\max\left(t(x), t_0\right)} + A \tag{5}$$

Where I_{inv}^{c} is the inverted image, A is the global atmospheric light, $t(x)$ is the transmission map. Image information will miss when transmission $t(x)$ tends to be zero. Therefore t_0 which is usually set to be 0.01 is used to limit the lower bound of the transmission.

F. Inverting image

The enhancement result can be obtained by inverting the scene radiance $J(x)$.

By analyzing the key steps mentioned above, we can draw a conclusion that the steps inverting input, estimating transmission, and de-hazing only operate on a single pixel, and there is none data dependency between each pixel. It makes these steps have high independence and maximum parallelism degree can reach the total number of pixels. The step of computing dark channel map needs to do minimum filter for three channels in a local patch. Although all pixels in one local patch are associated, we can create threads with pixel as unit, and each thread computes the dark channel value of one single pixel. The maximum parallelism degree can also reach the total number of pixels.

In the step of estimating atmospheric light, the method in [11] need to sort all the pixels in the dark channel map, and then select the 0.1% pixels which have the maximum intensity. Because of all the pixels having great data dependency in [11], it is not suitable for parallel computing. We need a new approach to estimate atmospheric light on GPU. The essence of estimating atmospheric light in dark channel prior is balancing the max luminance and the max dark pixels [12], so we estimate atmospheric light by evaluating the luminance and dark pixel for each pixel in proposed enhancement algorithm. The atmospheric light evaluation value $W(x)$ is given by

$$W(x) = \lambda \times Y(x) + (1-\lambda) \times D^{dark}(x) \tag{6}$$

Where $Y(x)$ is the luminance map, $D^{dark}(x)$ is the dark channel value, λ is the weight of luminance, which is calculated by

$$\lambda = D^{dark}(x)/(Y(x) + D^{dark}) \tag{7}$$

After obtained the evaluation value $W(x)$ of each pixel, we choose the max $W(x)$ as the global atmospheric light A. The proposed approach can choose the pixel with high luminance from the pixels with high dark channel value as the global atmospheric light. Because the step of evaluating atmospheric light value only operate on a single pixel, the maximum parallelism degree can reach total pixel numbers.

IV. The Implementation Of CUDA-Based Low Light Level Image Enhancement

The main idea of CUDA heterogeneous programming model is that the parts on host side (CPU) process in serial, and the parts on device side (GPU) process in parallel. To implement

the low light level image enhancement algorithm using CPU-GPU heterogeneous architecture, we execute the serial parts of the enhancement algorithm on CPU, mainly containing the read and write operations of the image. The data are copied from CPU memory to GPU memory firstly, and then the whole enhancement algorithm is implemented on GPU. Finally, the enhancement results are copied from GPU to CPU. As the transfer overhead between CPU and GPU occupies much execution time, all the data required are copied to the global memory on GPU one-time to increase the execution speed.

Because of the collaboration of CPU and GPU, there is transfer overhead caused by the calling and returning of kernels. Therefore in this paper, we merge the 6 key steps of the enhancement algorithm into 3 kernels. Kernel1 processes the step of inverting input, estimating transmission, and calculating the atmospheric light evaluation values. Kernel2 processes the step of estimating atmospheric light based on the result of kernel1. Kernel3 processes the step of estimating transmission, de-hazing and inverting image. The number of threads which created by kernels is the same as the total number of image pixels, and each thread processes only one single pixel. The flow chart of proposed enhancement approach is illustrated in Fig. 1.

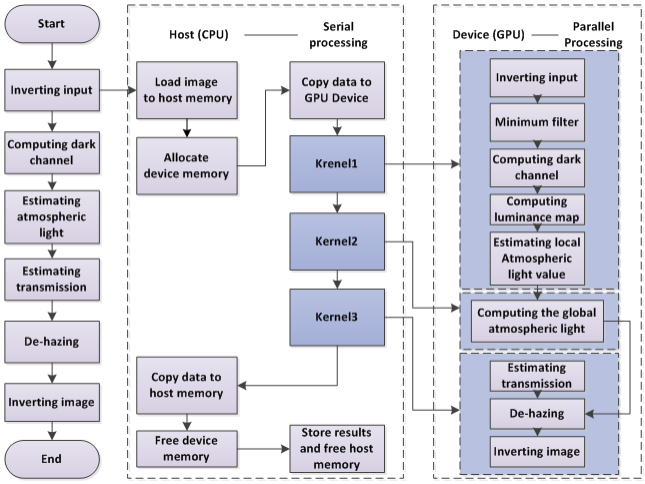

Fig. 1. The flow chart of proposed parallel enhancement approach

A. Kernel 1

The operation of inverting input is a single inversion for each pixel. Since the number of threads is limited, we create 512 threads in one block and $M \times N/512$ blocks is used for processing each color channel of an image with a size of $M \times N$. To achieve the maximum parallelism, one thread is used to do inversion operation for a single pixel.

Computing dark channel map is to do minimum filter for three color channels in a local patch, and then select the minimum value among the three color channels of the filter results. Experiments show that the window with a size of 7×7 can obtain outstanding performance. During the parallel implementation, the memory access to transfer the RGB values of the inverted image is necessary, and the process of comparing data of each pixel in the 7×7 size window makes massive global memory access overhead of the same pixel value. To avoid the time overhead caused by repeated global memory access, we select shared memory instead of global memory, and divide the data based on the predefined block before computing dark channel map. Because shared memory can be accessed by all threads in the same block with high processing speed, using shared memory can reduce the repeated memory access to global memory and improve the time performance.

Calculating atmospheric light evaluation value is based on luminance map and dark channel map using (6), (7). Each thread can compute the atmospheric light evaluation value W for one pixel. The evaluation values are stored in global memory for subsequent calculations. The process of kernel 1 is illustrated in Fig. 2.

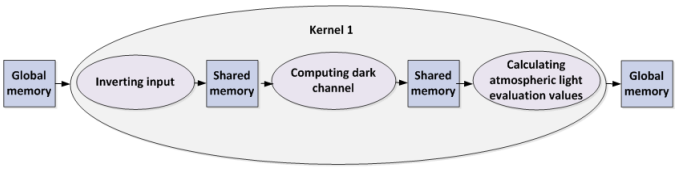

Fig. 2. The processing flow chart of kernel 1

B. Kernel 2

Kernel 2 is used to obtain the maximum atmospheric light evaluation value as the global atmospheric light based on the results of kernel1 stored in global memory. To obtain the maximum atmospheric light evaluation value from all pixels, we use the merge method. To implement the merge method, we need shared memory to store the intermediate result.

We first merge all the atmospheric light evaluation values W in each block, and store the maximum values of one block in shared memory.To get the maximum value W of all pixels, the block maximum values is merged in shared memory next, and the last maximum evaluation value W in shared memory is the global atmospheric light A. For n threads in one block, we call kernel 2 log (n) times to merge the corresponding atmospheric light evaluation value W to obtain the global atmospheric light. The merge method of calculating the maximum evaluation value is illustrated in Fig. 3.

C. Kernel 3

As the global memory transfer is one of the determinations of the effectiveness of the parallel implementation [13], we merge the last three steps into one kernel. Because of each pixel is none data dependency, pixels can be processed individually and the parallelism degree is the number of total pixels.

In Kernel 3, the image data of RGB three color channels are read from the global memory firstly, and then the luminance value is computed using (4). Secondly we compute the

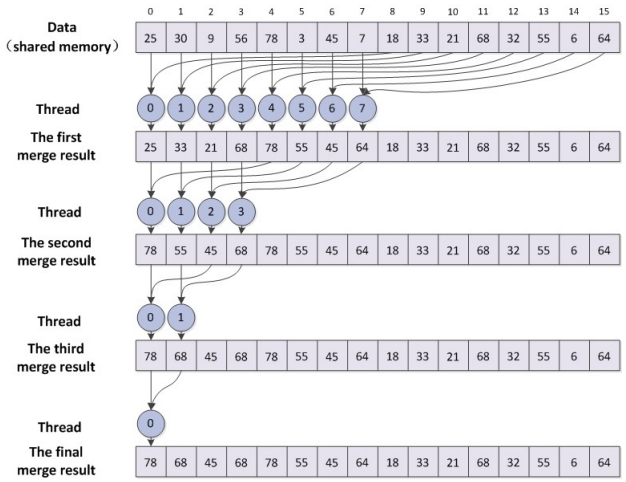

Fig. 3. Merge for the maximum value

transmission value of the current pixel using (3) and store it in shared memory. Thirdly each thread uses the de-hazing model to get the recovery image based on the transmission value in shared memory and the global atmospheric light value in global memory, and then store the results in shared memory. Finally, we invert the recovery image stored in shared memory to get the final enhancement results. The processing flow chart of kernel 3 is illustrated in Fig. 4.

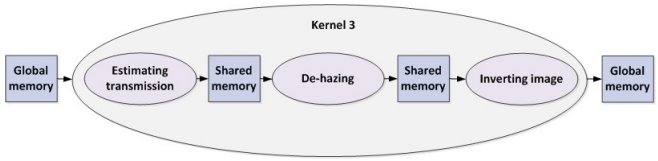

Fig. 4. The processing flow chart of kernel 3

V. Experimental Results

Our experiments of low light level enhancement approach are employed on NVIDIA GeForce GTX770 with 2GB global memory, and Intel Core i5-2400 CPU. Other configurations are windows7 operation system, runtime environment VS2012, and development environment CUDA toolkit 6.0.

A. The time performance

Table I shows the timing measurements of each step of the proposed approach and also for the overall execution on CPU and GPU for a 1920×1080 high-resolution image.

Table I illustrates that the runtime of each step on GPU has great speed improvement compared with CPU. As shown in Table I, the total time of each step in enhancement algorithm is only 11.88ms, but the overall execution time reaches 61ms. Although the proposed approach does the CPU-GPU data transfer only once, the memory allocation and transfer overhead occupies over 80% of the overall execution time. Fig. 5 intuitively compares the speedup of each step using histogram. As shown in Fig. 5, the speedup is obvious, but it makes a great difference between each step. The maximum speedup can achieve several hundreds of times, the minimum speedup is also more than tenfold. Because of all data are integer type, the speedup is high when computing inverted image and dark channel map. The speedup is low when estimating atmospheric light because the process has time overhead of the loop calling of kernel2 to obtain the maximum in shared memory. Multiple times of shared memory access make the speedup of estimating atmospheric light not so obvious. The speedup is also relatively low when estimating transmission and using de-hazing model because of a large scale of floating point calculation. Although the speedup makes difference, the overall execution has a significant speed improvement.

TABLE I
The Timing Measurements Of Proposed Algorithm

Step	CPU (ms)	GPU (ms)	Speedup
Inverting input	31	0.36	86
Computing dark channel	125	0.41	304
Estimating atmospheric light	78	4.73	71
Estimating transmission	31	2.73	11
De-hazing	94	3.65	18
Overall execution	734	61	12

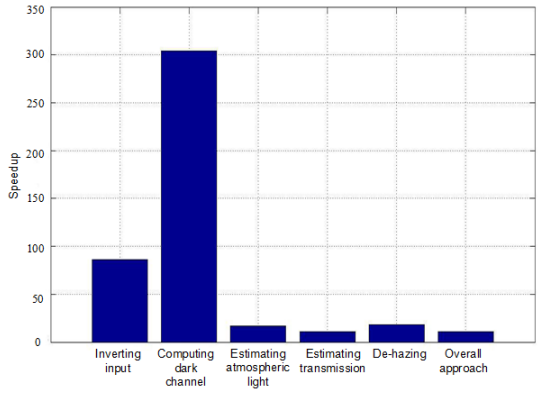

Fig. 5. The speedup of each step of the algorithm

Table II compares the runtime of proposed approach for images of different sizes based on GPU with CPU. It illustrates that the speedup increases as the size of images increases and conclusion can be made that however big the image size is, the acceleration effect is obvious on GPU. The runtime of a 1024×768 high-resolution image is just 23ms and the processing speed can reach 40 frames per second. The experiment results show that the proposed approach can achieve real-time effect for a high-resolution image.

In this paper, we use a new approach of estimating atmospheric light by using luminance map and dark channel map for parallel computing. The runtime of traditional enhancement approach and proposed enhancement approach on GPU for images with different sizes is shown in Table III. It illustrates that the runtime on GPU is approximately reduced by half using proposed approach. Because of the increasing of paral-

TABLE II
THE RUNTIME COMPARISON OF GPU TO CPU

Image size	CPU (ms)	GPU (ms)	Speedup
256 × 144	25	3	8
512 × 288	52	5	10.4
730 × 548	140	13	10.7
1024 × 768	265	23	11.5
1920 × 1080	734	61	13.0

lelism degree, the time performance of proposed enhancement algorithm has been significantly improved on GPU.

TABLE III
THE RUNTIME COMPARISON OF TRADITIONAL ALGORITHM WITH PROPOSED ALGORITHM ON GPU

Image size	Traditional algorithm(ms)	Proposed algorithm(ms)
256 × 144	4.56	2.79
512 × 288	10.59	5.05
730 × 548	23.69	12.88
1024 × 768	43.65	23.02
1920 × 1080	104.90	54.98

Using shared memory in thread block instead of global memory, the proposed method reduced the repeated access times of the same pixel. The speedup comparison of images with different size between proposed method and the method without using shared memory is shown in Fig. 6. As shown in Fig. 6, the proposed method using shared memory makes more markedly speedup.

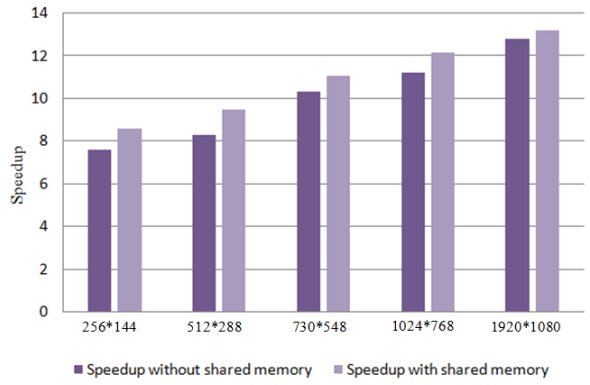

Fig. 6. The performance improvement using shared memory

VI. VISUAL EFFECT OF PROPOSED APPROACH

The visual effect of proposed enhancement method implementing on CPU-GPU heterogeneous architecture is shown in Fig. 7. Fig. 7(a) is the original low light level image. Only little scene information can be seen under the low light condition. The CUDA-based enhancement result using proposed method is shown in Fig. 7(b). Not only the surrounding scene information of car and wall are visible, some details as the license plate number can also be observed after processing.

Fig. 7. (a) The original low light level image (b) The enhancement result using proposed approach

VII. CONCLUSION

This paper studied the key steps of enhancement algorithm for low light level images, and proposed a CUDA-based parallel enhancement approach with improving the part of estimating atmospheric light. Three kernels are used to process the whole steps of the algorithm to reduce the transfer time overhead created by kernels. Shared memory and global memory are fully used to reduce the time overhead of memory access. Experimental results show that the proposed approach has a good effect in both visual quality and processing speed.

ACKNOWLEDGMENT

This project is supported by NSFC Grant (No.61305109, No.61072105), by 863 Program (2013AA014601), and by Shaanxi Scientific research plan (2013K06-09, 2014K07-11).

REFERENCES

[1] Yunbo Rao, Weiyao Lin, Leiting Chen, Image-based fusion for video enhancement of nighttime surveillance, Optical Engineering, 2011, 50(5):1-7
[2] Land, J.McCann, Ligtness and Retinex theory, J.Opt.Soc.Amer, Jan, 1971, vol.61(1), pp.1-11.
[3] Jobson, G.Woodell, Propertiesand performance of a center/surround Retinex, IEEE Trans, Image Process, 1997.3(6):451-462.
[4] Jobson, Z.Rahman, G.Woodell, A multiscale retinex for bridging the gap between color images and the human observation of scenes[J], IEEE Trans.Image Process.1997.6(7):965-976.
[5] X.Dong, G.Wang, Y.Pang et al, Fast Efficient Algorithm for Enhancement of Low Lighting Video[C], IEEE International Conference on Multimedia and Expo, 2011
[6] Xiang Dong Zhang, Peiyi Shen, Lingli luo.et al, Enhancement and Noise Reduction of Very Low Light Level Images, 21st. International Conference on Pattern Recognition, 2012.
[7] Niklas Perrersson, GPU-Accelerated real-time surveillance de-weathering.Computer vision laboratory, Department of electrical engineering, 2013
[8] Eric Olmedo,Jorge de la Calleja.et al, Point to point processing of digital images using parallel computing, International journal of computer science issues, ISSN:1698-0814, 2012.
[9] Chen guo Liang, Design and analysis of parallel algorithms, Beijing higher education press, 2002.
[10] NVIDIA.CUDA toolkit documentation, CUDA C programming guide changes from version 6.0, 2014.
[11] K.He, J.Sun, and X.Tang, Single image haze removal using dark channel prior, IEEE Conference on Computer Vision and pattern Recognition(CVPR,09), 2009.
[12] Yungang Xue, Jv Ren, Huayou Su et al., Parallel Implementation and Optimization of Dehazing Using Dark Channel Prior Based on CUDA, HPC China, 2012
[13] Kyu Park, Nitin Singhal, Man Hee Lee et al. Design and Performance Evaluation of Image Processing Algorithms on GPUs, IEEE Transcations on Parallel and Distributed Systems, 2011